

A Comprehensive Compendium of Agentic Coding Frameworks and Model Context Protocols (2026)

Section 1: Model Context Protocol (MCP) Architectures

Architectural Forward

The widespread adoption of the Model Context Protocol (MCP) marks a structural paradigm shift in artificial intelligence system design. Historically, context injection required monolithic application architectures where developers manually parsed external data and injected it directly into the Large Language Model's (LLM) prompt window. The evidence suggests that this approach scaled poorly and created severe security bottlenecks regarding credential management and data boundaries.

MCP fundamentally decouples the reasoning engine (the LLM client) from the data execution layer (the server). By standardizing communication over standard input/output (stdio) for local, ephemeral processes, and Server-Sent Events (SSE) over HTTP (streamable-http) for distributed networks, MCP establishes a universal bus for context routing. The underlying trend indicates a transition toward composable AI infrastructure: LLMs increasingly operate as pure, decentralized computational nodes that dynamically negotiate capabilities with peripheral servers through a standardized handshake.

The primary implementations leverage three distinct primitives: Tools (model-controlled functional execution endpoints), Resources (application-controlled contextual data retrieval), and Prompts (user-controlled interaction templates). The data below transcribes the exact current practices for both Python-based SDKs (including `mcp.server` and `FastMCP`) and strict, schema-validated TypeScript implementations utilizing the official `@modelcontextprotocol/sdk`.

[Entry 1.1] Python MCP SDK: Core Server and Client Implementation

Description: The official Python SDK provides the fundamental primitives for establishing an MCP server. It bypasses complex protocol handling and JSON schema generation by inspecting Python type hints and docstrings. The architecture allows clients to connect seamlessly over stdio or HTTP transports.

Transcription (Server and Client):

```
# Server Implementation (server.py)
from mcp.server import MCPServer

mcp = MCPServer("Demo")

@mcp.tool()
def add(a: int, b: int) -> int:
```

```

    """Add two numbers."""
    return a + b

@mcp.resource("greeting://{name}")
def greeting(name: str) -> str:
    """Greet someone by name."""
    return f"Hello, {name}!"

# Client Implementation (client.py)
import asyncio
from mcp import Client
from server import mcp

async def main() -> None:
    # Client connects directly to a server object in memory for local
    # execution
    async with Client(mcp) as client:
        result = await client.call_tool("add", {"a": 1, "b": 2})
        print(result.structured_content)  # Expected Output:
        {'result': 3}

if __name__ == "__main__":
    asyncio.run(main())

```

[Entry 1.2] Python FastMCP: Advanced Asynchronous Tools and Resources

Description: The FastMCP framework abstracts the lower-level SDK, heavily utilizing decorators to define tools, resources, and prompts while automatically managing progress reporting and asynchronous task execution. This paradigm forces the LLM to adhere to specific schema types and gracefully manages long-running agentic tasks.

Transcription (FastMCP Server):

```

from typing import Annotated
from mcp.server.fastmcp import Context, FastMCP
from mcp.server.session import ServerSession
from mcp.server.fastmcp.prompts import Message
from mcp.server.fastmcp.resources import ResourceResult,
ResourceContent
import asyncio
import json
import aiofiles
from datetime import datetime, timezone

# Initialize the FastMCP server with explicit HTTP transport bindings
mcp = FastMCP(
    name="FastMCP StreamableHTTP Demo",

```

```

        host="127.0.0.1",
        port=8765,
    )

@mcp.tool(timeout=30)
async def countdown(
    start: int,
    ctx: Annotated[Context[ServerSession, None], "Injected by
FastMCP"],
) -> list[int]:
    """Count down from start to zero, demonstrating async execution
and context injection."""
    result = []
    for i in range(start, -1, -1):
        result.append(i)
        await asyncio.sleep(1)
        if ctx and ctx.session:
            await ctx.session.send_progress_notification(
                progress=start - i,
                total=start,
                message=f"Countdown at {i}"
            )
    return result

@mcp.resource("data://users")
def get_users() -> ResourceResult:
    """Provides a complex resource containing multiple MIME types."""
    return ResourceResult(
        contents=[
            ResourceContent(content='[{"id": 1, "name": "Admin"}]',
mime_type="application/json"),
            ResourceContent(content="# Users Database\nAdmin user
details included.", mime_type="text/markdown"),
        ],
        meta={"total": 1}
    )

@mcp.resource("time://{zone}")
def get_time(zone: str) -> str:
    """Return ISO 8601 timestamp."""
    now = datetime.now(timezone.utc)
    return now.isoformat() if zone.lower() == "utc" else
now.astimezone().isoformat()

@mcp.prompt(
    name="analyze_data_request",
    description="Creates a request to analyze data with specific
parameters",

```

```

        tags={"analysis", "data"},
        meta={"version": "1.1", "author": "data-team"}
    )
def data_analysis_prompt(data_uri: str, method: str = "summary") ->
list[Message]:
    """Sets up a structured reasoning scenario with initial
messages."""
    return [
        Message(f"You are a principal data scientist. The dataset is
at: {data_uri}"),
        Message(f"Please perform a '{method}' analysis on this data.",
role="user")
    ]

if __name__ == "__main__":
    mcp.run(transport="stdio")

```

[Entry 1.3] TypeScript MCP SDK: Zod Schema Validation

Description: Enterprise TypeScript MCP environments mandate strict runtime validation using libraries such as Zod. This approach establishes rigid error boundaries for API calls and forces the LLM to structure its arguments according to exact schema properties before the server attempts to parse the payload.

Transcription (Node.js/TypeScript):

```

#!/usr/bin/env node
/**
 * MCP Server Implementation for TypeScript utilizing
@modelcontextprotocol/sdk.
 * Demonstrates runtime Zod input validation, strict typing, and
structured JSON output.
 */

import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { StreamableHTTPServerTransport } from
"@modelcontextprotocol/sdk/server/streamableHttp.js";
import { StdioServerTransport } from
"@modelcontextprotocol/sdk/server/stdio.js";
import express from "express";
import { z } from "zod";
import axios, { AxiosError } from "axios";

enum ResponseFormat {
    MARKDOWN = "markdown",
    JSON = "json"
}

const server = new McpServer({

```

```

    name: "example-mcp",
    version: "1.0.0"
  });

const UserSearchInputSchema = z.object({
  query: z.string()
    .min(2, "Query must be at least 2 characters")
    .max(200, "Query must not exceed 200 characters")
    .describe("Search string to match against names/emails"),
  limit: z.number()
    .int()
    .min(1)
    .max(100)
    .default(20)
    .describe("Maximum results to return"),
  offset: z.number()
    .int()
    .min(0)
    .default(0)
    .describe("Number of results to skip for pagination"),
  response_format: z.nativeEnum(ResponseFormat)
    .default(ResponseFormat.MARKDOWN)
    .describe("Output format: 'markdown' or 'json'")
}).strict();

type UserSearchInput = z.infer<typeof UserSearchInputSchema>;

server.registerTool(
  "example_search_users",
  {
    title: "Search Example Users",
    description: `Search for users in the Example system by name,
email, or team.
Args:
  - query (string): Search string to match
  - limit (number): Maximum results (default: 20)
  - offset (number): Pagination offset (default: 0)
  - response_format ('markdown' | 'json'): Output format`,
    inputSchema: UserSearchInputSchema,
    annotations: {
      readOnlyHint: true,
      destructiveHint: false,
      idempotentHint: true,
      openWorldHint: true
    }
  },
  async (params: UserSearchInput) => {
    try {

```

```

    const data = await makeApiRequest<any>("users/search", "GET",
undefined, {
    q: params.query,
    limit: params.limit,
    offset: params.offset
  });

  const users = data.users || [];
  const total = data.total || 0;

  if (!users.length) {
    return {
      content: [{ type: "text", text: `No users found matching
'${params.query}'` }],
    };
  }

  const output = {
    total,
    count: users.length,
    offset: params.offset,
    users: users.map((user: any) => ({
      id: user.id,
      name: user.name,
      email: user.email,
      ...(user.team ? { team: user.team } : {}),
      active: user.active ?? true
    })),
    has_more: total > params.offset + users.length
  };

  let textContent: string;
  if (params.response_format === ResponseFormat.MARKDOWN) {
    const lines = [`# User Search Results: '${params.query}'`, "",
`Found ${total} users`, ""];
    for (const user of users) {
      lines.push(`## ${user.name} (${user.id})`);
      lines.push(`- **Email**: ${user.email}`);
      if (user.team) lines.push(`- **Team**: ${user.team}`);
    }
    textContent = lines.join("\n");
  } else {
    textContent = JSON.stringify(output, null, 2);
  }

  return {
    content: [{ type: "text", text: textContent }],
    structuredContent: output
  }

```

```

    };
  } catch (error) {
    return {
      content: [{ type: "text", text: handleApiError(error) }]
    };
  }
}
);

async function makeApiRequest<T>(endpoint: string, method: string,
data?: any, params?: any): Promise<any> {
  return { users: [], total: 0 };
}

function handleApiError(error: unknown): string {
  if (axios.isAxiosError(error)) {
    return `API Error: ${error.response?.data?.message ||
error.message}`;
  }
  return error instanceof Error ? error.message : "An unknown error
occurred";
}

const transport = process.env.TRANSPORT || 'stdio';
if (transport === 'http') {
  const app = express();
  app.use(express.json());

  app.post('/mcp', async (req, res) => {
    const transport = new StreamableHTTPServerTransport({
      sessionIdGenerator: undefined,
      enableJsonResponse: true
    });
    res.on('close', () => transport.close());
    await server.connect(transport);
    await transport.handleRequest(req, res, req.body);
  });

  const port = parseInt(process.env.PORT || '3000');
  app.listen(port, () => console.error(`MCP server running on
http://localhost:${port}/mcp`));
} else {
  const transport = new StdioServerTransport();
  server.connect(transport).then(() => console.error("MCP server
running via stdio"));
}

```

Section 2: Eastern-Origin Orchestration Frameworks

Architectural Forward

While standard ReAct (Reasoning and Acting) loops dictate much of Western agentic orchestration, a divergence in framework architecture is distinctly visible in tools originating from Chinese artificial intelligence research centers. Frameworks such as DB-GPT, Qwen-Agent, MindSearch, and ModelScope-Agent replace probabilistic LLM self-routing with highly deterministic, graph-based (Directed Acyclic Graph) topologies and specialized multi-agent hierarchies.

The underlying trend indicates a move toward "Agentic Workflow Expression Languages" (AWEL) and pre-determined data pipelines. By treating the LLM not as an autonomous free-agent but as a specific computational node within a DAG—constrained by predefined input nodes and strict routing logic—these systems significantly reduce hallucinated tool invocations. They excel in environments demanding high data governance, such as enterprise databases or large-scale document parsing, where traditional free-roaming agents fail to maintain coherent state over long execution horizons.

[Entry 2.1] Qwen-Agent: Core Tool Registration and Code Interpreters

Description: Qwen-Agent, developed by Alibaba, encapsulates LLM instruction following, planning, and code interpreter functionalities optimized for the Qwen model lineage. It permits the rapid deployment of Assistant agents capable of executing Python code sandboxes and generating multi-modal UI elements via custom APIs. The framework provides atomic components such as prompts and high-level wrappers for DashScope model servers.

Transcription:

```
import os
import json
import json5
import urllib.parse
from qwen_agent.agents import Assistant
from qwen_agent.tools.base import BaseTool, register_tool
from qwen_agent.utils.output_beautify import typewriter_print

@register_tool('my_image_gen')
class MyImageGen(BaseTool):
    description = 'AI painting (image generation) service. Inputs text
description, returns drawn image URL.'
    parameters = [{
        'name': 'prompt',
        'type': 'string',
        'description': 'Detailed description of the desired image
content, in English',
        'required': True
    }]

    def call(self, params: str, **kwargs) -> str:
```



```

        prompt = json5.loads(params) ['prompt']
        prompt = urllib.parse.quote(prompt)
        return json.dumps(
            {'image_url':
f'https://image.pollinations.ai/prompt/{prompt}'},
            ensure_ascii=False
        )

llm_cfg = {
    'model': 'qwen3:8b',
    'model_server': 'http://localhost:11434/v1',
    'api_key': 'EMPTY',
    'generate_cfg': {
        'top_p': 0.8
    }
}

system_instruction = '''After receiving the user's request, you
should:
- first draw an image and obtain the image url,
- then run code `request.get(image_url)` to download the image,
- and finally select an image operation from the given document to
process the image.
Please show the image using `plt.show()`. '''

tools = ['my_image_gen', 'code_interpreter']
files = [os.path.abspath('image_processing_doc.pdf')]

bot = Assistant(
    llm=llm_cfg,
    system_message=system_instruction,
    function_list=tools,
    files=files
)

messages = []
while True:
    query = input('\nuser query: ')
    messages.append({'role': 'user', 'content': query})

    print('bot response:')
    for response in bot.run(messages=messages):
        print(response)

    messages.extend(response)

```

[Entry 2.2] DB-GPT: Agentic Workflow Expression Language (AWEL)

Description: DB-GPT is an AI-native data application framework that leverages AWEL to define complex, stateful workflows where models, prompt templates, and execution contexts are orchestrated through explicit DAGs. This paradigm shifts agentic coding toward a pipeline architecture, ensuring privacy through localized execution and deterministic routing.

Transcription (AWEL DAG Construction):

```
import asyncio
from dbgpt.core.awel import DAG, InputOperator
from dbgpt.core.operators import (
    PromptBuilderOperator,
    RequestBuilderOperator,
)
from dbgpt.model.proxy import OpenAILLMClient
from dbgpt.model.operators import LLMOperator
from dbgpt_ext.rag import ChunkParameters
from dbgpt_ext.rag.operators.db_schema import
DBSchemaAssemblerOperator
from dbgpt_ext.storage.vector_store.chroma_store import
ChromaVectorConfig, ChromaStore

with DAG("simple_sdk_llm_example_dag") as dag:
    prompt_task = PromptBuilderOperator(
        "Write a SQL of {dialect} to query all data of {table_name}."
    )
    model_pre_handle_task =
RequestBuilderOperator(model="gpt-3.5-turbo")
    llm_task = LLMOperator(OpenAILLMClient())

    prompt_task >> model_pre_handle_task >> llm_task

output = asyncio.run(
    llm_task.call({
        "dialect": "MySQL",
        "table_name": "users"
    })
)
print(output)
```

Transcription (Multi-Agent Conversation Loop):

```
import asyncio
import os
from dbgpt.agent import AgentContext, AgentMemory, LLMConfig,
UserProxyAgent
from dbgpt.agent.expand.code_assistant_agent import CodeAssistantAgent
from dbgpt.model.proxy import OpenAILLMClient
```

```

llm_client = OpenAILLMClient(
    model_alias="gpt-4o",
    api_base=os.getenv("OPENAI_API_BASE"),
    api_key=os.getenv("OPENAI_API_KEY"),
)

async def main():
    context = AgentContext(conv_id="test123", language="en",
temperature=0.5, max_new_tokens=2048)
    agent_memory = AgentMemory()

    coder = (
        await CodeAssistantAgent()
        .bind(context)
        .bind(LLMConfig(llm_client=llm_client))
        .bind(agent_memory)
        .build()
    )

    agent_memory.gpts_memory.init(conv_id="test123")
    user_proxy = await
UserProxyAgent().bind(context).bind(agent_memory).build()

    await user_proxy.initiate_chat(
        recipient=coder,
        reviewer=user_proxy,
        message="Calculate the result of 321 * 123 using Python code
execution."
    )

if __name__ == "__main__":
    asyncio.run(main())

```

[Entry 2.3] MindSearch (InternLM): Cognitive Graph Mapping

Description: MindSearch emulates human cognitive information retrieval by dividing the AI architecture into a high-level WebPlanner and numerous parallel WebSearchers. The planner decomposes user queries into atomic sub-questions, outputting executable graph-construction code. This allows the framework to aggressively scale context gathering (often parsing over 300 web pages concurrently) without overflowing standard LLM context windows, returning highly factual and deep reasoning summaries.

Transcription:

```

import os
from mindsearch.agent.streaming import AsyncMindSearchAgent,
MindSearchAgent
from mindsearch.agent.models import GPTAPI

```

```

# Configure the reasoning engine via a SiliconFlow Proxy or direct API
internlm_silicon = dict(
    type=GPTAPI,
    model_type='internlm/internlm2_5-7b-chat',
    key=os.environ.get('SILICON_API_KEY', 'YOUR SILICON API KEY'),
    openai_api_base='https://api.siliconflow.com/v1/chat/completions',
    meta_template=[
        dict(role='system', api_role='system'),
        dict(role='user', api_role='user'),
        dict(role='assistant', api_role='assistant'),
        dict(role='environment', api_role='system')
    ],
    top_p=0.8,
    top_k=1,
    temperature=0,
    max_new_tokens=8192,
    repetition_penalty=1.02,
    stop_words=['<|im_end|>']
)

use_async = True
lang = "en"
agent_cls = AsyncMindSearchAgent if use_async else MindSearchAgent

# InterpreterParser resolves the LLM output into discrete topological
steps
agent = agent_cls(
    llm=internlm_silicon,
    template="date_template",
    output_format=InterpreterParser(
        begin="<|action_start|><|interpreter|>",
        end="<|action_end|>",
        template=(GRAPH_PROMPT_EN + graph_fewshot_example_en)
    )
)

async def generate(inputs, session_id):
    """Execution loop where WebPlanner dispatches queries to
    WebSearchers"""
    async for message in agent(inputs, session_id=session_id):
        yield message

```

[Entry 2.4] ModelScope-Agent: Unified Extensible Tooling Hub

Description: The ModelScope-Agent framework ties seamlessly into the broader ModelScope ecosystem, offering a low-coupling mechanism to bind custom tools (e.g., Code Interpreters, Web Browsers, Text-to-Speech) to localized models. It provides a direct pipeline for converting

and testing models inside a secure execution environment, establishing MCP links for external resolution.

Transcription:

```
import os
import asyncio
from ms_agent import LLMAgent
from modelscope_agent.tools import ModelscopePipelineTool
from modelscope.utils.constant import Tasks
from modelscope_agent.output_wrapper import AudioWrapper

# Custom Tool Registration within the ModelScope ecosystem
class TexttoSpeechTool(ModelscopePipelineTool):
    default_model = 'damo/speech_sambert-hifigan_tts_zh-cn_16k'
    description = 'Text-to-speech service'
    name = 'modelscope_speech-generation'
    parameters: list = [{
        'name': 'input',
        'description': 'Text to convert to speech',
        'required': True
    }, {
        'name': 'gender',
        'description': 'User identity (male/female)',
        'required': True
    }]
    task = Tasks.text_to_speech

    def _remote_parse_input(self, *args, **kwargs):
        pass

# MCP Server Configuration bridging to ModelScope inference services
mcp_config = {
    "mcpServers": {
        "fetch": {
            "type": "sse",
            "url":
"https://{your_mcp_url}.api-inference.modelscope.net/sse"
        }
    }
}

async def main():
    # Initialize agent pointing to the MCP configuration
    llm_agent = LLMAgent(mcp_config=mcp_config)

    # The agent routes the query through the MCP configuration to
    locate the appropriate schema
    await llm_agent.run('Briefly introduce modelscope.cn')
```

```
if __name__ == '__main__':
    asyncio.run(main())
```

Section 3: Visual Workflow & Tool Configuration Schemas

Architectural Forward

Enterprise platforms like Dify abstract Python and Node.js execution behind visual node-based canvases. However, the linkage between the visual interface and the underlying execution engine relies on exceptionally strict YAML definitions. These files define properties, metadata, credential requirements, and precise execution topologies.

A defining feature of this architecture is the rigid schema applied to error handling. If a code node executes an LLM-generated script that raises an exception, the YAML configuration dictates alternative execution paths to ensure the agentic workflow does not fatally crash. The underlying framework logic dynamically validates input credentials against these schemas prior to permitting model instantiation.

Error Strategy	Description	Structural Requirement
fail-branch	Creates an alternative execution path on error, providing error_message.	Uses sourceHandle: "fail-branch" in edge configuration. Requires variable-aggregator.
default-value	Returns predefined fallback value on error.	Continues main execution path; bypasses branching.
abort	Stops entire execution (default behavior).	None.
retry	Triggers predefined retry loop.	Defined in retry_config section of node properties.

[Entry 3.1] Dify Tool Configuration and Custom Provider YAML

Description: The transcription below outlines how a custom API or database provider (e.g., Anthropic, Weaviate) is defined within Dify using YAML schemas, followed by the Python class validation that enforces strict output boundaries.

Transcription (Provider Configuration - YAML):

```
provider: anthropic
label: Anthropic API
description: Access Claude models via official API.
icon: icon.svg
background: "#FFF"
help:
  title: Get your API Key
  url: https://console.anthropic.com/keys
supported_model_types:
  - llm
configure_methods:
```

- predefined-model

```
provider_credential_schema:
  type: object
  properties:
    anthropic_api_key:
      type: string
      label: API Key
      required: true
      secret: true

error_handling:
  strategy: fail-branch
  retry_config:
    max_retries: 3
    interval: 2000
```

Transcription (Provider Validation - Python):

```
from dify_plugin import ModelProvider
from dify_plugin.errors import CredentialsValidateFailedError
from dify_plugin.models import ModelType
import logging

logger = logging.getLogger(__name__)

class AnthropicProvider(ModelProvider):
    def validate_provider_credentials(self, credentials: dict) ->
None:
    """
    Validates provider credentials against the YAML schema
    properties.
    :param credentials: Provider credentials as defined in the
    YAML schema
    :raises CredentialsValidateFailedError: If validation fails
    """
    try:
        model_instance = self.get_model_instance(ModelType.LLM)
        model_instance.validate_credentials(
            model="claude-3-opus-20240229",
            credentials=credentials
        )
    except CredentialsValidateFailedError as ex:
        raise ex
    except Exception as ex:
        logger.exception(f"{self.get_provider_schema().provider}
credentials validate failed")
        raise ex
```

Section 4: Frontier Model Tool Schemas and Reasoning Streams

Architectural Forward

As autonomous systems increase in complexity, relying purely on natural language output for downstream orchestration proves brittle. Provider SDKs have resolved this by strictly standardizing function schemas (binding models to OpenAPI-like JSON structures) and exposing the internal reasoning streams of the LLM.

For Google's Gemini, the FunctionDeclaration structure dictates exactly how the model perceives available external tooling. By utilizing distinct configuration modes (AUTO, ANY, NONE), developers restrict the model's output bounds, forcing it into deterministic API interactions rather than generalized conversational responses. When parallel function calling occurs, the client iterates over the generated arguments and executes the logic before returning the combined state payload to the LLM.

Simultaneously, the DeepSeek API exposes a crucial mechanistic feature: the reasoning_content stream. In multi-turn tool-calling environments, caching and parsing the Chain-of-Thought (CoT) independently of the final tool parameter generation is essential. If the reasoning stream triggers safety filters or logical inconsistencies, external middleware can intercept the execution before the action_input is ever sent to a sandboxed environment.

[Entry 4.1] Google GenAI SDK: Function Declarations and Execution Constraints

Description: Google's google-genai Python library handles both automatic and manual function resolution. Manual resolution requires explicit parsing of the function_calls response object, affording the system strict control over execution lifecycles, loop limits, and tool authorization.

Transcription:

```
from google import genai
from google.genai import types

client = genai.Client(api_key="YOUR_GEMINI_API_KEY")

# 1. Manual Function Declaration
get_weather_func = types.FunctionDeclaration(
    name="get_current_weather",
    description="Get the current weather conditions for a given city.",
    parameters={
        "type": "object",
        "properties": {
            "location": {
                "type": "string",
                "description": "The city and state, e.g. San
```



```

    "unit": {
        "type": "string",
        "enum": ["celsius", "fahrenheit"],
        "description": "Temperature unit"
    },
    "required": ["location"]
}
)

# 2. Configure Model Execution Constraints
config = types.GenerateContentConfig(
    tools=[get_weather_func],
    automatic_function_calling=types.FunctionCallingConfig(
        disable=True,
        maximum_remote_calls=2
    ),
    tool_config=types.ToolConfig(
        function_calling_config=types.FunctionCallingConfig(mode='ANY')
    )
)

user_prompt_content = types.Content(
    role="user",
    parts=[types.Part.from_text(text="What is the weather like in
Boston?")],
)

response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=[user_prompt_content],
    config=config,
)

# 3. Execution Interception and Resolution
if response.function_calls:
    for call in response.function_calls:
        print(f"Model requests execution of: {call.name} with
parameters: {call.args}")

        # Developer executes local code here, obtaining a result
        dictionary
        mock_result = {"temperature": 72, "conditions": "sunny"}

        # Inject tool response back into the context cycle

```

```

        function_response_part = types.Part.from_function_response(
            name=call.name,
            response=mock_result
        )

```

[Entry 4.2] DeepSeek API: Explicit Reasoning Stream Parsing

Description: In applications utilizing DeepSeek models, the underlying Chain-of-Thought logic is separated from the final semantic or structural output. The reasoning_content delta allows agents to observe the model's internal plan before finalizing the tool execution strings.

Transcription:

```

from openai import OpenAI
import os

client = OpenAI(
    api_key=os.environ.get("DEEPSEEK_API_KEY"),
    base_url="https://api.deepseek.com/v1"
)

response = client.chat.completions.create(
    model="deepseek-reasoner",
    messages=[
        {"role": "user", "content": "Analyze the log file anomalies
and determine the required CLI tool."}
    ],
    stream=True
)

reasoning_log = ""
final_answer = ""

for chunk in response:
    # Capture the Chain-of-Thought logic prior to output generation
    if chunk.choices[0].delta.reasoning_content:
        reasoning_log += chunk.choices[0].delta.reasoning_content
        print(f"Thought: {chunk.choices[0].delta.reasoning_content}",
end="", flush=True)

    # Capture the finalized instruction or JSON payload
    elif chunk.choices[0].delta.content:
        final_answer += chunk.choices[0].delta.content
        print(chunk.choices[0].delta.content, end="", flush=True)

print("\n\n--- Full Reasoning Context ---")
print(reasoning_log)

```

Section 5: Context Management & Compression Middleware

Architectural Forward

As autonomous systems aggregate data from web searchers, databases, and continuous ReAct loops, context bloat becomes computationally unviable. The causal relationship is direct: excessively long prompts degrade model attention, increase token latency, and balloon inference costs. Context management middleware, such as LLMLingua, utilizes selective token filtration and dynamic reordering algorithms to compress prompts before they hit the final inference endpoint. This is critical for agents acting as final summarizers in multi-agent arrays, preventing performance degradation over long-running background tasks.

[Entry 5.1] LLMLingua Prompt Compression Integration

Description: LLMLingua acts as a smart middle layer, compressing retrieved context blocks directly. It integrates cleanly with retrieval pipelines to optimize the prompt budget dynamically based on configured compression ratios and sorting algorithms.

Transcription:

```
from llmlingua import PromptCompressor
from langchain_community.document_compressors import
LLMLinguaCompressor

# Standard Initialization of the compression engine
llm_lingua = PromptCompressor()

prompt_list = [
    "Extensive document content containing hundreds of pages of
regulatory compliance data...",
    "Secondary document containing redundant legal boilerplate..."
]

# Compress raw context prior to LLM injection
compressed_prompt = llm_lingua.compress_prompt(
    prompt_list,
    question="What are the main regulatory changes in the last
quarter?",
    rate=0.55,
    condition_in_question="after_condition",
    reorder_context="sort",
    dynamic_context_compression_ratio=0.3,
    condition_compare=True,
    context_budget="+100",
    rank_method="longllmlingua"
)
```

```
print("Compressed Prompt:", compressed_prompt['compressed_prompt'])
print("Tokens Saved:", compressed_prompt['compressed_tokens'])
```

Section 6: Secure Agentic Sandboxing & Execution Environments

Architectural Forward

The fundamental vulnerability of agentic frameworks lies in code execution: permitting an autonomous entity to write and execute arbitrary Python or bash scripts on a host machine poses catastrophic security risks. As agents gain access to persistent filesystems to read repositories or manipulate datasets, the potential for destructive commands (e.g., recursive deletion) or credential exfiltration necessitates robust containerization. Modern implementations employ "sandboxes all the way down," leveraging Docker, Kubernetes, or microVM architectures like gVisor (runsc). These environments mount project directories ephemerally, strip elevated privileges (no sudo), and manage I/O streams programmatically. The frameworks below outline standard procedures for instantiating and controlling these dockerized lifecycles within Python code, reflecting varying tiers of security versus execution overhead.

Framework / Feature	eval() / exec()	Safe-Py-Runner	Docker Sandbox
Speed to Setup	Instant	Seconds	Minutes
Overhead	None	Very Low	High
Security Mechanism	None	Policy-Based limits, AST checks	Isolated Container / microVM
Optimal Use Case	Local testing only	Fast AI Prototyping / Internal tools	Production-scale agentic execution

[Entry 6.1] Docker-Backed Sandbox Runner (OpenAI Agents)

Description: This implementation explicitly orchestrates the lifecycle of a Docker-backed sandbox session. It injects a file manifest, instantiates a tool call interface for the agent to inspect the workspace, and streams the output directly to the terminal.

Transcription:

```
import argparse
import asyncio
import sys
from pathlib import Path
from docker import from_env as docker_from_env

from openai.types.responses import ResponseTextDeltaEvent
from agents import ModelSettings, Runner
from agents.run import RunConfig
from agents.sandbox import SandboxAgent, SandboxRunConfig
from agents.sandbox.config import DEFAULT_PYTHON_SANDBOX_IMAGE
from agents.sandbox.sandboxes.docker import DockerSandboxClient,
DockerSandboxClientOptions
```

```

from examples.sandbox.misc.example_support import text_manifest,
tool_call_name
from examples.sandbox.misc.workspace_shell import
WorkspaceShellCapability

DEFAULT_QUESTION = "Summarize this sandbox project in 2 sentences. Do
not guess file names."

async def main():
    # 1. Build a manifest mapping host files to the sandbox workspace
    manifest = text_manifest({
        "README.md": "# Demo Project\nThis sandbox contains a tiny
demo project.\n",
        "src/app.py": 'def greet(name: str) -> str:\n    return
f"Hello, {name}!"\n',
        "docs/notes.md": "# Notes\nWorkspace manipulation
demonstration."
    })

    # 2. Configure Agent with access to the Docker workspace shell
    agent = SandboxAgent(
        name="Docker Workspace Explorer",
        instructions="Inspect the project before answering. Keep the
response concise.",
        capabilities=[WorkspaceShellCapability()],
        model_settings=ModelSettings(tool_choice="required"),
        default_manifest=manifest
    )

    # 3. Instantiate the container lifecycle
    docker_client = DockerSandboxClient(docker_from_env())
    sandbox = await docker_client.create(
        manifest=manifest,

options=DockerSandboxClientOptions(image=DEFAULT_PYTHON_SANDBOX_IMAGE)
,
    )

    try:
        # async with maintains the session lifecycle connection
        async with sandbox:
            print(f"Executing Query: {DEFAULT_QUESTION}\n")
            result = Runner.run_streamed(
                agent,
                DEFAULT_QUESTION,

run_config=RunConfig(sandbox=SandboxRunConfig(session=sandbox)),
    )

```

```

        # Process output streams
        saw_text_delta = False
        for event in result:
            if isinstance(event, ResponseTextDeltaEvent):
                saw_text_delta = True
                print(event.delta, end="", flush=True)
            elif hasattr(event, "tool_call"):
                print(f"\n[Tool Execution Intercepted]:
{tool_call_name(event)}")

        if saw_text_delta:
            print("\n")

    finally:
        await sandbox.close()

if __name__ == "__main__":
    asyncio.run(main())

```

[Entry 6.2] All-in-One (AIO) Agent Sandbox Client

Description: The AIO Sandbox pushes the boundary by merging browser interaction (CDP/VNC), Jupyter notebook execution, and shell operations into a single Docker instance exposed via RESTful APIs. This prevents the fragmentation of files that occurs when separate sandboxes are used for web crawling and code execution, allowing files downloaded by an agentic browser tool to be immediately parsed by a python execution node.

Transcription:

```

from agent_sandbox import Sandbox

# Initialize client pointing to the exposed Docker container port
# Note: Docker run command requires `--security-opt seccomp=unconfined`
# -p 127.0.0.1:8080:8080`
client = Sandbox(base_url="http://localhost:8080")

# Access the isolated environment's directory structure
home_dir = client.sandbox.get_context().home_dir

# 1. Execute shell commands safely within the container
result = client.shell.exec_command(command="ls -la")
print("Shell Output:", result.data.output)

# 2. Cross-domain File Operations (e.g., processing files downloaded
via browser)
write_status = client.file.write(
    filepath=f"{home_dir}/agent_output.txt",
    content="Agent execution completed successfully."
)

```

```

)
print("File Written:", write_status)

# 3. Read modified states
content = client.file.read(filepath=f"{home_dir}/agent_output.txt")
print("File Content:", content.data)

# 4. Integrate with Jupyter for stateful Python execution
jupyter_result = client.jupyter.execute(
    code="""
import pandas as pd
df = pd.DataFrame({'Data': [1, 2, 3]})
print(df.head())
"""
)
print("Jupyter Execution Result:", jupyter_result.output)

```

[Entry 6.3] SandboxAI: Lightweight Client Execution

Description: For environments seeking to avoid heavy Docker-in-Docker overhead, SandboxAI offers a lightweight client implementation that utilizes an embedded server to manage localized containers. It provides both full-agency (root container access) and limited-agency (specific command isolation) execution.

Transcription:

```

from sandboxai import Sandbox

# The embedded=True parameter spins up a local management server
with Sandbox(embedded=True) as box:
    # 1. Direct Python Execution
    python_result = box.run_ipython_cell("print('Hello from the Python Sandbox')")
    print(python_result.output)

    # 2. Direct Shell Command Execution
    shell_result = box.run_shell_command("ls /")
    print(shell_result.output)

```

Works cited

1. Model Context Protocol - GitHub, <https://github.com/modelcontextprotocol> 2. Build an MCP Server in TypeScript: A Production Guide, <https://theroadtoenterprise.com/blog/build-mcp-server-typescript> 3. Practical Guide to MCP (Model Context Protocol) in Python - DEV Community, https://dev.to/m_sea_bass/practical-guide-to-mcp-model-context-protocol-in-python-ijd 4. Building a Simple MCP Server in Python Using the MCP Python SDK - GitHub, <https://github.com/ruslanmv/Simple-MCP-Server-with-Python> 5. The official Python SDK for

Model Context Protocol servers and clients - GitHub,
<https://github.com/modelcontextprotocol/python-sdk> 6. The FastMCP Client,
<https://gofastmcp.com/clients/client> 7. Prompts - FastMCP,
<https://gofastmcp.com/servers/prompts> 8. Resources & Templates - FastMCP,
<https://gofastmcp.com/servers/resources> 9. Node/TypeScript MCP Server Implementation Guide
- GitHub,
https://github.com/anthropics/skills/blob/main/skills/mcp-builder/reference/node_mcp_server.md
10. Demonstration of DB-GPT: Next Generation Data Interaction System Empowered by Large
Language Models - arXiv, <https://arxiv.org/html/2404.10209v3> 11. MindSearch {CJK}UTF8gbsn
思·索: Mimicking Human Minds Elicits Deep AI Searcher - arXiv,
<https://arxiv.org/html/2407.20183v1> 12. MindSearch: Mimicking Human Minds Elicits Deep AI
Searcher - arXiv, <https://arxiv.org/html/2407.20183v2> 13. Data Driven Multi-Agents - DB-GPT,
<http://docs.dbgpt.cn/docs/agents/introduction/> 14. Use Data App With AWEL - DB-GPT,
<http://docs.dbgpt.cn/docs/application/awel/> 15. Blog - DB-GPT, <http://docs.dbgpt.cn/blog/> 16.
QuickStart Basic AWEL Workflow - eosphoros-ai/DB-GPT - GitHub,
https://github.com/eosphoros-ai/DB-GPT/blob/main/docs/docs/awel/cookbook/quickstart_basic_awel_workflow.md 17. Qwen-Agent - Read the Docs,
https://qwen.readthedocs.io/en/v2.0/framework/qwen_agent.html 18. Qwen-Agent: A Guide
With Demo Project - DataCamp, <https://www.datacamp.com/tutorial/qwen-agent> 19. Deploying
AI Agents Locally with Qwen3, Qwen-Agent, and Ollama | by Benjamin Consolvo | Intel Tech |
Medium,
<https://medium.com/intel-tech/deploying-ai-agents-locally-with-qwen3-qwen-agent-and-ollama-cad452f20be5> 20. modelscope-agent - PyPI, <https://pypi.org/project/modelscope-agent/> 21.
modelscope-agent - CodeSandbox,
<https://codesandbox.io/p/github/MarleneJiang/modelscope-agent> 22. dify-workflow-skills -
LobeHub, https://lobehub.com/skills/neversight-skills_feed-dify-workflow-skills 23. Model
Provider Plugin - Dify Docs,
<https://docs.dify.ai/en/develop-plugin/dev-guides-and-walkthroughs/creating-new-model-provider>
24. weaviate - Dify Marketplace, <https://marketplace.dify.ai/plugin/weaviate/weaviate> 25. How to
Implement Function Calling with Gemini for Tool-Augmented AI Applications,
<https://oneuptime.com/blog/post/2026-02-17-how-to-implement-function-calling-with-gemini-for-tool-augmented-ai-applications/view> 26. Function calling reference | Gemini Enterprise Agent
Platform | Google Cloud Documentation,
<https://docs.cloud.google.com/gemini-enterprise-agent-platform/reference/models/function-calling>
27. Thinking Mode - DeepSeek API Docs,
https://api-docs.deepseek.com/guides/thinking_mode/ 28. Function Calling - Google Gen AI
Python SDK, <https://googleapis-python-genai-70.mintlify.app/guides/function-calling> 29. Gemini
API: Function calling with Python - Google Colab,
https://colab.research.google.com/github/google-gemini/cookbook/blob/main/quickstarts/Function_calling.ipynb 30. DeepSeek API Tutorial: Low-Cost AI in Python (2026) | FreeAPIHub,
<https://freeapihub.com/blog/deepseek-api-tutorial-python-low-cost-ai> 31. How to Compress Your
Prompts and Reduce LLM Costs - DEV Community,
<https://dev.to/manishmshiva/how-to-compress-your-prompts-and-reduce-llm-costs-3pa> 32.
LLMLingua/examples/Code.ipynb at main - GitHub,
<https://github.com/microsoft/LLMLingua/blob/main/examples/Code.ipynb> 33. GitHub -
zigotica/agent-sandbox: Docker-based sandbox that runs AI coding agents with no git, no SSH,
and can only access project folder. Built because permission prompts aren't real protection —
you end up accepting everything by reflex, that's not a decision, it's fatigue. pi and opencode

built-in. Extendable to use other agents running a single register command.,
<https://github.com/zigotica/agent-sandbox> 34. safe-py-runner: Secure & lightweight Python execution for LLM Agents - Reddit,
https://www.reddit.com/r/Python/comments/1rejn2o/safepyrunner_secure_lightweight_python_execution/ 35. Code execution capability (sandboxed) · Issue #33 · pydantic/pydantic-ai-harness - GitHub, <https://github.com/pydantic/pydantic-ai-harness/issues/33> 36. All-in-One Sandbox for AI Agents that combines Browser, Shell, File, MCP and VSCode Server in a single Docker container. - GitHub, <https://github.com/agent-infra/sandbox> 37. Secure Code Execution for the Age of Autonomous AI Agents | by Vlad Kolesnikov | Google Cloud - Medium,
<https://medium.com/google-cloud/secure-code-execution-for-the-age-of-autonomous-ai-agents-d52e7acd6c5d> 38. openai-agents-python/examples/sandbox/docker/docker_runner.py at main - GitHub,
https://github.com/openai/openai-agents-python/blob/main/examples/sandbox/docker/docker_runner.py 39. GitHub - substratusai/sandboxai: Run AI generated code in isolated sandboxes,
<https://github.com/substratusai/sandboxai>